> ## ⚠ SUPERSEDED — DO NOT QUOTE THIS DOCUMENT This report's conclusion is **the opposite of the project's current finding**, and the difference is a measurement instrument, not a reanalysis. Everything below uses the **single-core** compute microbenchmark. On that signal the analytical roofline formula genuinely did lose to the `1/bandwidth_ratio` baseline, and the "a single hardware ratio is the state of the art" conclusion was correct *for that signal*. Addition 4 replaced the microbenchmark with a **multi-core aggregate** probe. The compute signal's correlation with scaling went from **+0.213 (physically wrong sign)** to **−0.691 (correct sign)**, and the analytical roofline then beat the baseline by a bootstrap-significant margin.
> `scripts/analysis/fix3_parallelism_mechanism.py` confirmed the mechanism causally (r = 0.458, p = 0.037): PostgreSQL chooses parallel plans, so the compute resource actually consumed is *aggregate* multi-core throughput, which the single-core probe could not see. Two further defects in the numbers below, both fixed in the canonical pipeline: 1. They were computed on the **buffer-bug dataset** (`data/raw_buggy_do_not_use/`). 2. The `p_*` and `analytical_scaling` features came from **one globally-fit classifier applied to held-out machines** — a leak. See §2 of `results/CANONICAL.md` . **For every current number, read `CANONICAL.md` , generated by `make results` .** This file is retained only as a record of what the single-core signal supported, and because its Limitations section is still accurate and still worth reading.

Cross-Environment TPC-H Runtime Prediction: Final Results and Limitations

Executive summary

On this dataset (5 AWS EC2 instance types, 21 TPC-H queries, 420 ordered dev→prod pairs), **a single hardware ratio — `1/bandwidth_ratio` — is the best-performing scaling-factor estimator, and no more sophisticated approach (bottleneck-weighted physics formula, bottleneck-gated formula, or a learned gradient-boosted quantile model) improves on it by a statistically significant margin under leave-one-machine-out cross-validation.** The gradient-boosted model's apparent variability across held-out machines is largely explained by one finding: it interpolates between the hardware profiles it was trained on rather than truly extrapolating to new hardware, while the physics-based formulas degrade far more gracefully on the one machine (c5a) that is a genuine outlier in hardware-signature space. This is the headline scientific claim of this work. The compute-side hardware signal remains too weak to safely incorporate, even after attempting to rescue it via bottleneck-gating. The learned p99 model is unsuitable as a standalone SLA gate (recall 0.985, precision 0.153); a three-state gate combining it with the analytical p50 estimate does not resolve this cleanly — see limitations below.

Final comparison table (leave-one-machine-out CV, MAPE on scaling_factor_p50, 2000-resample cluster bootstrap 95% CI)

approach	MAPE %	95% CI
naive (assume prod = dev)	31.97	[25.43, 39.89]
naive-linear (1/bandwidth_ratio)	**23.03**	**[20.29, 26.05]**
analytical roofline (bottleneck-weighted)	31.14	[29.79, 32.63]
bottleneck-gated (compute_ratio when p_compute≥0.5, else bandwidth_ratio)	28.06	[26.95, 29.18]
learned quantile GBR (p50)	31.26	[21.07, 43.32]

The bootstrap resamples at the machine (cluster) level, respecting the leave-one-machine-out grouping structure rather than treating individual rows as independent — the CI width for the learned model ([21.1, 43.3]) is itself evidence of high fold-to-fold variance: this model's apparent performance depends heavily on which machine happens to be held out.

Significance tests (paired cluster bootstrap on the MAPE difference):

- naive-linear vs. learned GBR: point estimate −8.24 pp, 95% CI **[−22.39, 2.41]** → **includes zero, not statistically significant.** We cannot claim the learned model is worse, but we equally cannot claim it is better or even competitive with confidence — it is statistically indistinguishable from noise around naive-linear on this sample size.
- naive-linear vs. analytical roofline: point estimate −8.11 pp, 95% CI **[−11.82, −4.40]** → **excludes zero, naive-linear significantly beats the analytical formula.** Blending in the weak compute signal measurably hurts, not just fails to help.

Bottom line: on this dataset, a single well-chosen hardware ratio is the state of the art. Learned and hybrid-physics methods do not yet justify their added complexity.

Headline finding: interpolation, not extrapolation

held-out machine	n	learned GBR MAPE %	analytical MAPE %	naive-linear MAPE %
c5a	168	55.54	33.89	19.66
c7i	168	34.17	29.63	27.68
m5a	168	27.02	31.83	20.57
r5n	168	23.78	30.97	25.98
z1d	168	15.79	29.35	21.25

c5a has the most extreme hardware signature in the set (bandwidth 14.9, the highest; compute 0.593, the lowest) — it sits furthest from the other four machines in signature space. When c5a is held out, the learned model's error more than triples relative to its best fold (55.5% vs 15.8%), while the analytical and naive-linear formulas barely move (33.9% and 19.7%, close to their cross-machine averages). Since the analytical and naive-linear approaches need no training examples of a machine to use its

(measured) hardware ratio, this asymmetry is direct evidence that **the learned model is interpolating between the hardware profiles of the four machines it did see, not extrapolating to genuinely unseen hardware** — exactly the failure mode a 5-machine, 5-fold evaluation is positioned to expose but not to fully characterize (see limitations).

Task A: attempting to repair the compute signal

subset	n pairs	compute_ratio corr.	bandwidth_ratio corr.
pooled	420	0.168	−0.638
compute-labeled queries (Q1, Q12, Q16)	60	0.306	−0.699
bandwidth-labeled queries (Q6, Q13, Q14, Q19)	80	0.278	—
io-labeled queries	80	−0.159	—
mixed-labeled queries	200	0.198	—

Isolating the compute-labeled queries does raise compute_ratio's correlation with scaling (0.168 → 0.306), confirming the pooling dilutes a real but weak signal. However, two things undercut using this as a fix: (1) bandwidth_ratio's correlation is *also* higher on compute-labeled queries than pooled (−0.699 vs −0.638) — bandwidth dominates even on queries we labeled as compute-bound, meaning the compute microbenchmark just isn't capturing what actually differentiates these machines' behavior on these queries; (2) compute_ratio's correlation on io-labeled queries is *negative* (−0.159), the wrong sign, meaning the gate's fallback behavior would need to be reliable everywhere it doesn't fire, and the underlying signal is inconsistent in direction across classes, not just noisy in magnitude.

Verdict: the bottleneck-gated formula (route to compute_ratio when p_compute ≥ 0.5, else bandwidth_ratio) does NOT beat the flat single-ratio baseline — 28.06% MAPE vs. 23.03% for naive-linear. Gating made things worse, not better. The diagnosis in Phase 6 stands: the compute microbenchmark (a scalar accumulation loop) does not produce a hardware signature that is usable even when restricted to the queries it should theoretically help most.

Calibration (single-sided coverage, from Phase 6, unchanged by Phase 7)

alpha	target	empirical coverage	nominal
0.50	scaling_factor_p50	48.3%	50%
0.95	scaling_factor_p95	80.7%	95%
0.99	scaling_factor_p99	94.4%	99%

p50 is well-calibrated. Both upper quantiles under-cover, and this is a small-sample effect, not (necessarily) a modeling defect: extreme quantiles (95th, 99th percentile) are the hardest statistics to estimate reliably from ~336 training rows per fold, since by definition only a handful of training examples inform the tail behavior the model is being asked to predict. **This should be read as "insufficient data to calibrate tail quantiles reliably," not "the quantile regression approach is fundamentally miscalibrated."**

Task C: three-state SLA gate

Gate logic: **block** if the analytical p50 estimate alone already exceeds the SLA threshold (scaling_factor > 1.5); **pass** if the high-recall learned p99 model predicts no breach; **warn** (defer to a real staging run) otherwise.

state	n	% of total	actual breach rate within state
block	66	7.9%	45.5%
warn	770	91.7%	12.9%
pass	4	0.5%	25.0%

Block-state error rate (blocked pairs that were *not* actually a breach): **54.5%**. Pass-state error rate (passed pairs that *were* actually a breach): **25.0%**, on only 4 pairs total.

This three-state gate does not resolve the precision/recall tradeoff cleanly, and reporting that plainly is more useful than presenting it as a solved problem. The root cause is the same one identified in Phase 6: the learned p99 model over-predicts breach almost everywhere (recall 0.985 came at the cost of flagging nearly every pair), so the "pass" zone — which requires the p99 model to actively clear a pair — is nearly empty (4 of 840 evaluated rows). The gate effectively collapses to "block roughly 8% of pairs outright, defer everything else to a real run," which is a legitimate and safe posture for an SLA gate (it never silently green-lights a risky deployment) but is not the crisp three-way triage the design aimed for. The block decision itself is also weak (54.5% of blocks are false alarms), so even the "confident" state should be treated as a prioritization signal for review, not an automatic hard stop.

Deployment recommendation: given the asymmetric cost of a missed SLA breach vs. a false alarm, we would deploy **naive-linear (1/bandwidth_ratio) as the point estimate** for scaling-factor prediction (best MAPE, tightest CI, statistically indistinguishable from every more complex alternative), paired with the **learned p99 model used only as a conservative flag, not a gate** — i.e., any pair the p99 model marks as risky routes to mandatory human review or a real staging benchmark before deployment, accepting a high false-alarm rate as the price of a low false-negative rate. We would not deploy the analytical roofline formula, the bottleneck-gated formula, or the three-state auto-gate as currently built; none earned their complexity over the flat ratio in this evaluation.

Limitations

- **5 machines.** Leave-one-machine-out CV here means 5 folds. This is enough to detect the interpolation-vs-extrapolation asymmetry (the c5a result), but not enough to bound how *often* a held-out machine will behave like c5a rather than like the other four — with one outlier machine out of five, we have a single data point demonstrating the failure mode exists, not a distribution characterizing its frequency.
- **EBS-only storage, no I/O-type diversity.** All five instance types (c5a, c7i, m5a, r5n, z1d) use network-attached EBS storage under the same AMI. The dataset cannot speak to how these predictions would behave on instance-local NVMe, different storage tiers, or storage-bound

workloads more broadly — the `io` bottleneck class in this project reflects buffer-touch patterns within a uniform storage backend, not genuine storage-hardware diversity.

- **Compute microbenchmark weakness.** The per-machine `compute` signature is a single scalar accumulation loop's timing, run once per machine. It correlates weakly (0.17 pooled) and inconsistently in direction (positive on compute/bandwidth/mixed queries, negative on io queries) with actual scaling behavior. Task A's attempt to rescue it via bottleneck-gating failed. Any future work replacing this with a richer microbenchmark suite (e.g., separate integer/float/vectorized throughput, cache-hierarchy-aware probes) is likely to matter more for overall accuracy than further modeling changes on top of the current signal.
- **Q15 excluded.** TPC-H Q15 requires a view/temp-table construct that needed special handling in the collection script and was dropped from this project's scope entirely (per the dataset collection guide). All 21-query statistics, correlations, and models in this project exclude it; no scaling behavior for Q15 is characterized here.
- **io/mixed bottleneck conflation (from Phase 5).** The bottleneck classifier's largest confusion is io queries being predicted as mixed (10 of 20 io-labeled rows in Phase 5's CV). Since the Phase 6/7 analytical and gated formulas use these probabilities as weights, any systematic io→mixed misclassification pushes weight toward the geometric-mean "mixed" term instead of the bandwidth-like io term, which is a plausible contributor to the analytical formula's underperformance beyond the compute-signal weakness alone — we have not isolated how much of the roofline formula's gap to naive-linear is attributable to this vs. to the compute signal itself, and disentangling the two would require either fixing Phase 5's io/mixed boundary or an ablation neither of which was in scope here.
- **Small absolute sample for tail statistics.** 21 queries × 5 machines × 20 repeats gives robust p50 estimates but thin support for p95/p99 percentiles of each query's own runtime distribution (each is estimated from only 20 raw runs), before any cross-machine modeling is layered on top. The calibration under-coverage at p95/p99 is consistent with, though not proven to be solely caused by, this upstream data thinness.
- **Physical-machine noise, not fully characterized post-hoc.** The dataset collection guide specifies a per-machine CV noise check (reject any instance with timing CV > 5%) before collection began, which controls for gross host contention at collection time, but no CV/noise metric was carried into the final dataset or used as a feature or filter in Phases 5–7 — we cannot retroactively distinguish "genuine hardware-driven scaling variance" from "residual measurement noise below the 5% collection-time threshold" in any of the results above.

Artifacts

- `models/phase7_gate_config.json` — three-state gate threshold configuration.
- `results/phase7_mape_bootstrap_table.csv` — final comparison table with bootstrap CIs.
- `results/phase7_gate_evaluation.csv` — three-state gate counts and error rates.
- `results/phase7_per_machine_breakdown.csv` — per-held-out-machine MAPE for all three formula-based/learned approaches.
- `results/phase7_compute_signal_diagnosis.csv` — compute_ratio / bandwidth_ratio correlations by bottleneck class.